

Guía de desarrollo y estrategia TDD

Ingeniería de Sistemas e Informática · Universidad Continental
Expediente organizado v1.0 | 22/09/2026

Preparación y ciclo

Entorno

Abrir TecnicoYa.code-workspace. Instalar dependencias de cada módulo según manifiestos y documentación técnica; copiar .env.example a configuración local y nunca versionar secretos. Verificar que las reglas coincidan mediante scripts/sync-rules.py --check.

Rojo

Partir de una historia y criterio. Escribir una prueba pequeña con entrada, acción y esperado. Ejecutarla y comprobar que falla por la conducta faltante, no por importación rota o entorno incorrecto. Guardar la identificación del caso.

Verde

Implementar el mínimo que cumple el criterio y ejecutar de nuevo. No endurecer datos solo para satisfacer el test ni cambiar el esperado para ocultar un defecto. Mantener permisos en el servidor.

Refactorizar

Mejorar nombres, duplicación y responsabilidades sin cambiar conducta. Repetir tests. Revisar contratos, migraciones y reglas compartidas. Un refactor no agrega alcance sin historia.

Límite histórico

Las pruebas existentes son base de regresión. No se afirma que todo el código anterior fue creado con TDD. Para nuevas tareas conservar evidencia del ciclo y revisión.

Ejemplos de TDD del dominio

Crterios y pruebas

Historia	Prueba inicial	Implementación esperada
Respuesta de oferta	A los 10 min exactos aceptar devuelve conflicto y no asigna.	Comparar tiempo servidor con expires_at antes de transacción final.
Matching	Dos candidatos con score igual se ordenan por mejor rating.	Orden secundario determinista; fixture común.
Privacidad	Cliente B cancela ID de A y obtiene prohibición.	Autorizar propiedad antes de modificar estado.
Calificación	Segunda nota no cambia promedio ni reviews.	Rechazar duplicado de forma atómica.
Frontend	Error HTTP conserva dirección y descripción.	Estado de campos separado del estado de envío.

Niveles de dobles

Mock de red sirve para error de interfaz; fake HTTP valida fallback de Laravel; no demuestra conexión Python real. Usar integración aparte con PostgreSQL y FastAPI. Controlar reloj para límites; no esperar 24 horas reales en pruebas unitarias.

Casos concurrentes

Crear test de dos aceptaciones sobre una solicitud en PostgreSQL aislado. Esperado: una propuesta ganadora y ninguna reserva duplicada. Verificar resultado persistido, no solo código HTTP.

Comandos y revisión

Ejecución desde cada módulo

Carpeta	Comando	Resultado a guardar
frontend/web	npm test	Suite y casos
frontend/web	npm run build	Build o error
backend/api	php artisan test	Casos y entorno de base
python/api	python -m pytest -q	Casos de ranking y validación
Raíz	python scripts/sync-rules.py --check	Consistencia de reglas

Revisión de código

Verificar alcance, legibilidad, autorización, datos, errores, pruebas significativas y ausencia de secretos. Revisar que los tests fallarían ante un defecto real. Evitar pruebas que solo repiten la implementación sin evaluar comportamiento.

Evidencia

Guardar comando, fecha, build/commit, entorno y salida íntegra. Vincular CT o CNF; registrar defectos y retest por separado. No afirmar cobertura de líneas o ramas sin reporte de herramienta y denominador definido.

Fuentes

Base documental: fuentes/proyecto-final.pdf; fuentes/plan-de-pruebas-original.pdf; fuentes/ejemplos-practica.docx; fuentes/estado-del-arte-original.pdf. Código: rutas relativas a la raíz del repositorio. Fuentes conservadas con SHA-256 en fuentes/manifest.json. Guía técnica del repositorio: docs/TECNICOYA.md.